Single View Pose Estimation

Prof. Ziyun (Claude) Wang
Department of Electrical and Computer Engineering
Spring 2026

Recap: projective transformations

A **projective transformation** is any invertible matrix transformation

$$\mathbb{P}^2 \to \mathbb{P}^2.$$

- A projective transformation $\mathbf{H}$ maps a point $\boldsymbol{p}$ to $\boldsymbol{p}' \sim \mathbf{H}\boldsymbol{p}$.
- Invertibility means $\det(\mathbf{H}) \neq 0$ and that there exists $\lambda \neq 0$ such that

$$\lambda \boldsymbol{p}' = \mathbf{H}\boldsymbol{p}.$$

- We will write either $\boldsymbol{p}' \sim \mathbf{H}\boldsymbol{p}$ or $\lambda \boldsymbol{p}' = \mathbf{H}\boldsymbol{p}$.

Recap: Direct Linear Transform (DLT)

Per-point block

For correspondence i with $(x_i, y_i) \leftrightarrow (u_i, v_i)$, define

$$\mathbf{A}_i = \begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -u_i x_i & -u_i y_i & -u_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -v_i x_i & -v_i y_i & -v_i \end{bmatrix} \in \mathbb{R}^{2\times 9}.$$

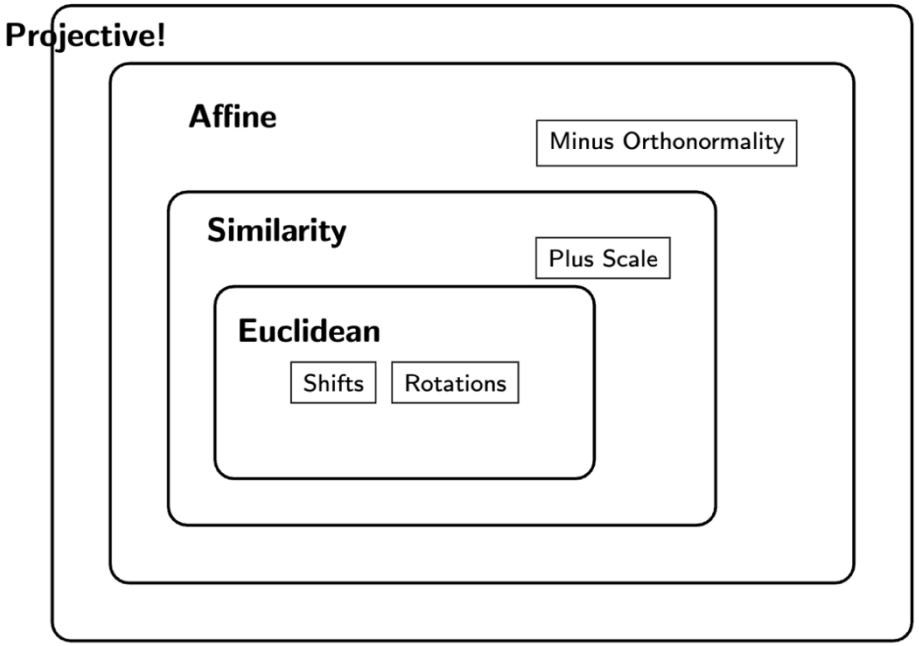
Stack all points

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \\ \vdots \\ \mathbf{A}_N \end{bmatrix} \in \mathbb{R}^{2N\times 9}, \qquad \mathbf{A}\boldsymbol{h} = \mathbf{0}.$$

Since $\boldsymbol{h}$ is defined up to scale, solve

$$\min_{\|\boldsymbol{h}\|=1} \|\mathbf{A}\boldsymbol{h}\|^2 \quad \Rightarrow \quad \boldsymbol{h} = \text{right singular vector of } \mathbf{A} \text{ with smallest singular value.}$$

Recap: Hierarchy of transforms



- **Euclidean (Rigid)**
  Preserves: Lengths, Areas, Angles.
- **Similarity**
  Preserves: Angles, Parallelism.
- **Affine**
  Preserves: Parallelism, Ratio of Areas.
- **Projective (Homography)**
  Preserves: Collinearity, Cross-ratio.

Camera Pose Estimation from a Single View

Plane-induced Homography

Plane parameterization

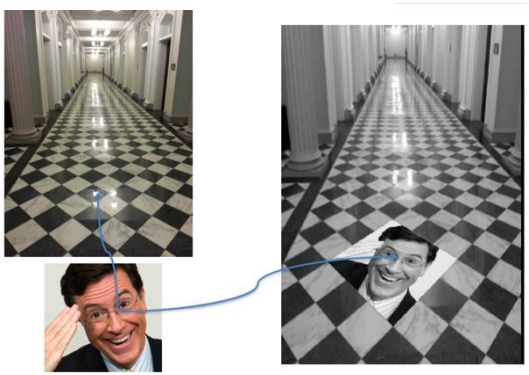
Choose plane coordinates $\boldsymbol{X}_\pi = (X, Y, 1)^\top$ (e.g., plane is $Z = 0$ in its own frame).

Induced homography

For points on the plane,

$$\lambda\, \boldsymbol{x} \;=\; \mathbf{H}\, \boldsymbol{X}_\pi, \qquad \boldsymbol{x} \in \mathbb{P}^2,\ \mathbf{H} \in \mathbb{R}^{3\times 3}.$$

Interpretation: a *single* 3×3 matrix explains all correspondences on the plane.



Pose from homography

Camera projection restricted to the plane

For a point $\boldsymbol{X} = (X, Y, 0)^\top$,

$$\lambda \boldsymbol{x} = \mathbf{K}\,[\mathbf{R}\,|\mathbf{t}] \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \mathbf{K}\,\left[\,\mathbf{R}_1\ \mathbf{R}_2\ \mathbf{t}\,\right] \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}.$$

So the homography is

$$\mathbf{H} = \mathbf{K}\,\left[\,\mathbf{R}_1\ \mathbf{R}_2\ \mathbf{t}\,\right],$$

where $\mathbf{R}_1, \mathbf{R}_2$ are the first two columns of $\mathbf{R}$.

Decompose H into (R, t)

Normalize

$$\widehat{\mathbf{H}} = \mathbf{K}^{-1}\mathbf{H} = \left[\, \hat{\boldsymbol{h}}_1 \;\, \hat{\boldsymbol{h}}_2 \;\, \hat{\boldsymbol{h}}_3 \,\right].$$

Recover pose up to a scale

Choose scale s (one common choice):

$$s = \frac{1}{\|\hat{\boldsymbol{h}}_1\|} \quad (\approx \frac{1}{\|\hat{\boldsymbol{h}}_2\|}).$$

Then

$$\boxed{\mathbf{R}_1 = s\,\hat{\boldsymbol{h}}_1,} \; \boxed{\mathbf{R}_2 = s\,\hat{\boldsymbol{h}}_2,} \quad \mathbf{t} = s\,\hat{\boldsymbol{h}}_3, \quad \mathbf{R}_3 = \mathbf{R}_1 \times \mathbf{R}_2.$$

x axis y axis z axis

Find the closest rotation

Because $\mathbf{H}$ is estimated from noisy data, the recovered columns are not perfectly orthonormal.

Project to the closest rotation

Form $\widetilde{\mathbf{R}} = [\mathbf{R}_1\ \mathbf{R}_2\ \mathbf{R}_3]$ and compute SVD:

$$\mathbf{R} = \mathbf{U}\,\mathrm{diag}(1, 1, \det(\mathbf{U}\mathbf{V}^\top))\,\mathbf{V}^\top.$$

Makes R a proper rotation with det(R) = 1

Cheirality (positive depth)

Pick the sign/scale so that the tag's known 3D points (on the plane $Z = 0$) transformed by the candidate pose

$$\boldsymbol{X}_c = \mathbf{R}\boldsymbol{X} + \mathbf{t}$$

This is because (R, t) and (-R, -t) give the same homography

Recover translation scale

- Scale can be recovered from the average scale of two rotation axis
- Key idea: global scale is the same for all three columns

$$\widehat{\mathbf{H}} = \mathbf{K}^{-1}\mathbf{H} = \left[\, \hat{\boldsymbol{h}}_1 \;\, \hat{\boldsymbol{h}}_2 \;\, \hat{\boldsymbol{h}}_3 \,\right]. \qquad s \approx \frac{1}{2}\left(\frac{1}{\|\hat{\boldsymbol{h}}_1\|} + \frac{1}{\|\hat{\boldsymbol{h}}_2\|}\right)$$

Then

$$\mathbf{R}_1 = s\,\hat{\boldsymbol{h}}_1, \quad \mathbf{R}_2 = s\,\hat{\boldsymbol{h}}_2, \quad \mathbf{t} = s\,\hat{\boldsymbol{h}}_3,$$

April Tags

An AprilTag detector typically returns the 4 image corners (pixel coordinates) and an ID.

Known geometry

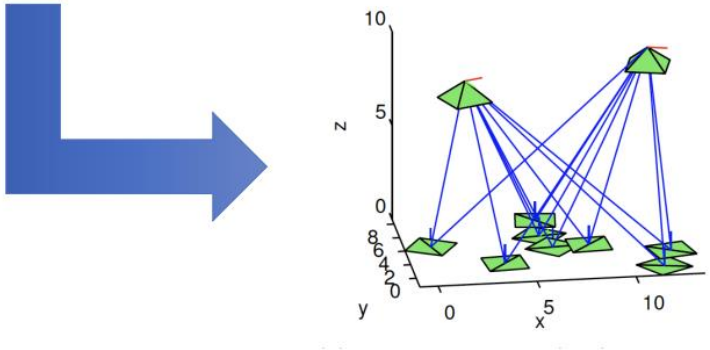
Given tag size L (meters), define tag-frame 3D corners (planar):

$$\boldsymbol{X}_1 = (-\tfrac{L}{2}, -\tfrac{L}{2}, 0)^\top,\ \boldsymbol{X}_2 = (\tfrac{L}{2}, -\tfrac{L}{2}, 0)^\top,\ \boldsymbol{X}_3 = (\tfrac{L}{2}, \tfrac{L}{2}, 0)^\top,\ \boldsymbol{X}_4 = (-\tfrac{L}{2}, \tfrac{L}{2}, 0)^\top.$$

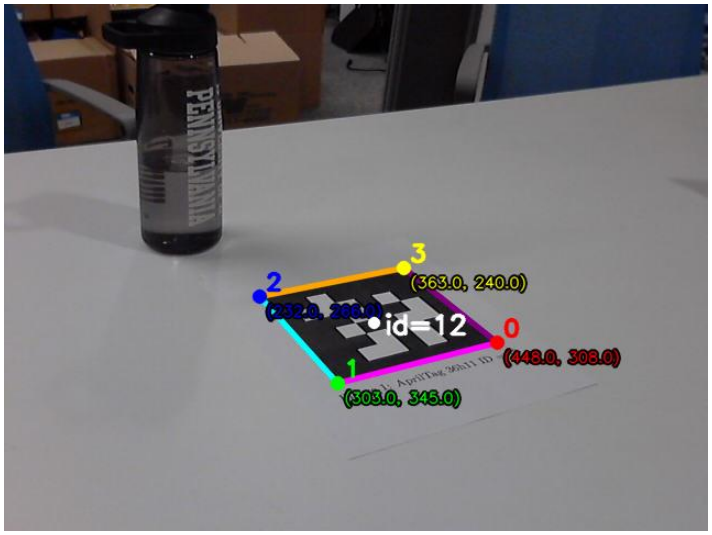
Measurements

Four 2D pixels (u_i, v_i) corresponding to those corners.

Pose of robot with respect to a planar surface



Example: pose from April Tags



Homography

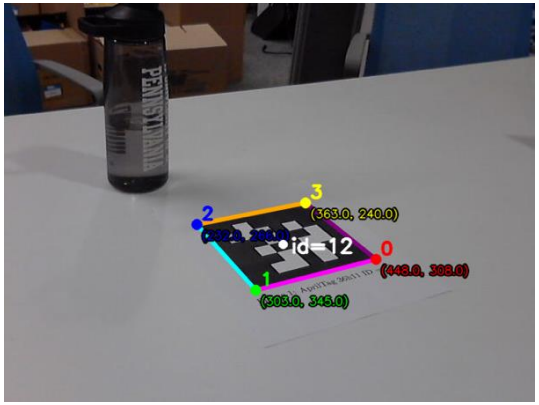
World measurement

Assume a tag of side length $L = 0.10\,\mathrm{m}$ (10 cm). In the tag frame ($Z = 0$), use the corner order top-left (TL), top-right (TR), bottom-right (BR), bottom-left (BL):

$$\boldsymbol{X}_{\mathrm{TL}} = (-0.05,\ \ 0.05, 0)^\top, \quad \boldsymbol{X}_{\mathrm{TR}} = (\ \ 0.05,\ \ 0.05, 0)^\top,$$
$$\boldsymbol{X}_{\mathrm{BR}} = (\ \ 0.05, -0.05, 0)^\top, \quad \boldsymbol{X}_{\mathrm{BL}} = (-0.05, -0.05, 0)^\top.$$

Pixel measurement

corner	(X, Y) on tag plane (m)	(u, v) in image (px)
TL	$(-0.050,\ \ 0.050)$	$(448,\ 308)$
TR	$(\ \ 0.050,\ \ 0.050)$	$(303,\ 345)$
BR	$(\ \ 0.050, -0.050)$	$(232,\ 266)$
BL	$(-0.050, -0.050)$	$(363,\ 240)$



Pose from homography

Assume the camera intrinsics are known. For a 640×480 image, take the principal point at the center and focal length $f = 800$ px:

$$\mathbf{K} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{H} \sim \begin{bmatrix} -1662.385690 & 279.042853 & 336.528571 \\ 63.214285 & 298.714281 & 286.325000 \\ -0.854762 & -1.497619 & 1 \end{bmatrix}$$

Step 1: normalize the homography

Compute $\widehat{\mathbf{H}} = \mathbf{K}^{-1}\mathbf{H} = [\hat{\boldsymbol{h}}_1 \ \hat{\boldsymbol{h}}_2 \ \hat{\boldsymbol{h}}_3]$. For the example above:

$$\widehat{\mathbf{H}} = \mathbf{K}^{-1}\mathbf{H} = \begin{bmatrix} -1.736077 & 0.947851 & 0.020661 \\ 0.335446 & 0.822679 & 0.057906 \\ -0.854762 & -1.497619 & 1 \end{bmatrix}.$$

Pick a scale s so that the first two columns have unit norm (average both for robustness):

$$s \approx \tfrac{1}{2}\left(\frac{1}{\|\hat{\boldsymbol{h}}_1\|} + \frac{1}{\|\hat{\boldsymbol{h}}_2\|}\right) \approx 0.510475.$$

Recover pose

From the scaled columns, form the *raw* rotation estimate

$$\widetilde{\mathbf{R}} = [\mathbf{R}_1 \ \mathbf{R}_2 \ \mathbf{R}_3].$$

For this example (computed from $s\widehat{\mathbf{H}}$):

$$\widetilde{\mathbf{R}} \approx \begin{bmatrix} -0.886112 & 0.483905 & 0.030494 \\ 0.171252 & 0.419991 & -0.477176 \\ -0.436466 & -0.764019 & -0.454107 \end{bmatrix}$$

From the example run (SVD of $\widetilde{\mathbf{R}}$):

$$\mathbf{U} \approx \begin{bmatrix} -0.982043 & -0.052345 & 0.181249 \\ -0.128541 & 0.888874 & -0.439749 \\ 0.138089 & 0.455151 & 0.879642 \end{bmatrix}, \qquad \mathbf{\Sigma} \approx \mathrm{diag}(1.011871,\ 0.999735,\ 0.988006),$$

$$\mathbf{V} \approx \begin{bmatrix} 0.778802 & 0 & -0.627270 \\ -0.627270 & 0 & -0.778802 \\ 0 & -1 & 0 \end{bmatrix}.$$

Recover pose

Pose (tag $\rightarrow$ camera) from SVD-orthonormalized rotation:

$$\mathbf{R} \approx \begin{bmatrix} -0.878509 & 0.474849 & 0.052345 \\ 0.175734 & 0.423107 & -0.888874 \\ -0.444229 & -0.771685 & -0.455151 \end{bmatrix}, \qquad \mathbf{t} \approx \begin{bmatrix} 0.010547 \\ 0.029560 \\ 0.510475 \end{bmatrix} \text{ m.}$$

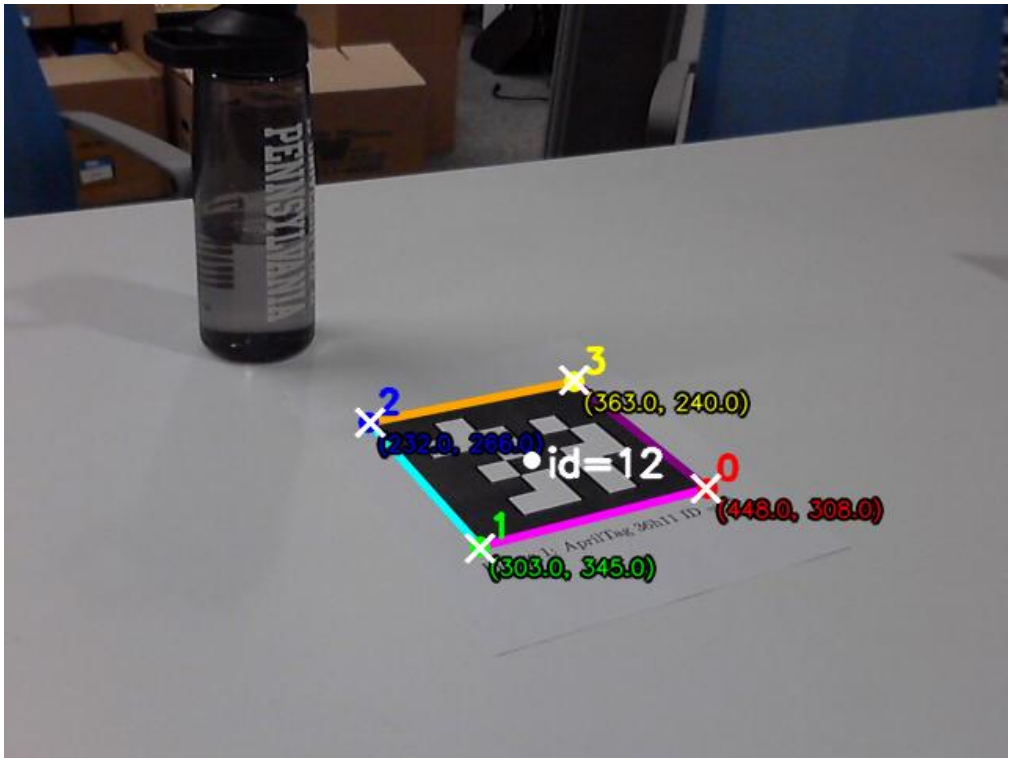
Sanity check

Project the 4 tag corners back to pixels. Reprojection RMSE: ≈ 1.063 px.

Not Great! Why?

Non-planar, wrong detection, motion-blur, wrong calibration

Reprojection Check



What if we have non-planar scenes?

Perspective N points (PnP)

Solve for camera pose if we know:

- Calibrated intrinsics $\mathbf{K}$ (or normalized image coordinates)
- n correspondences: 3D points $\boldsymbol{X}_i \in \mathbb{R}^3$ and 2D pixels $\boldsymbol{x}_i = (u_i, v_i)$

Output

A pose $(\mathbf{R}, \mathbf{t})$ that best explains the projections.

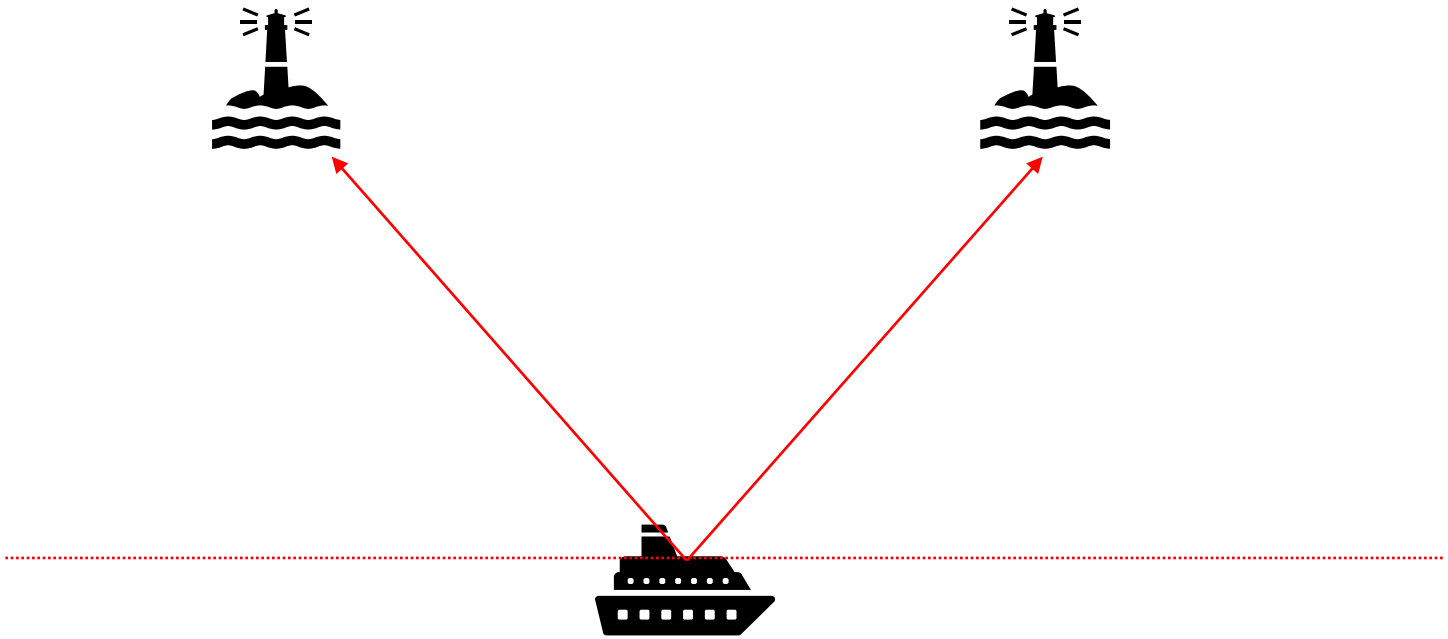
PnP is a core building block in SLAM, AR, marker tracking, and robotics.

Bearing Problem

- Reminder: a pixel is just a ray in the camera frame
- How do I know my location if I <u>only know the direction</u> of two known lighthouses?

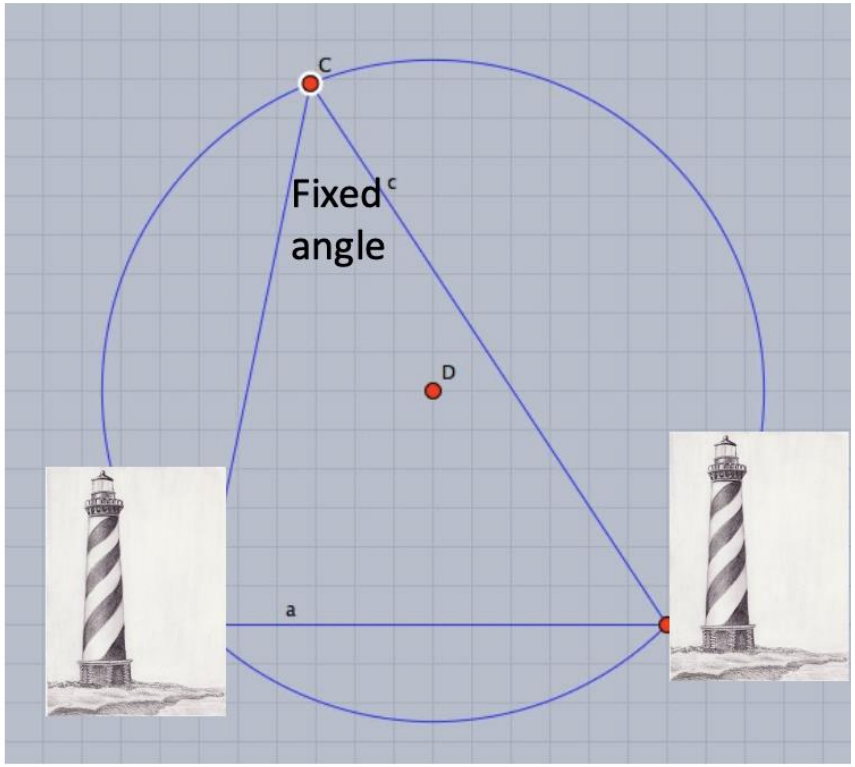
Bearing problem: lighthouse

Known beacon locations



Know angle between two lighthouses in camera

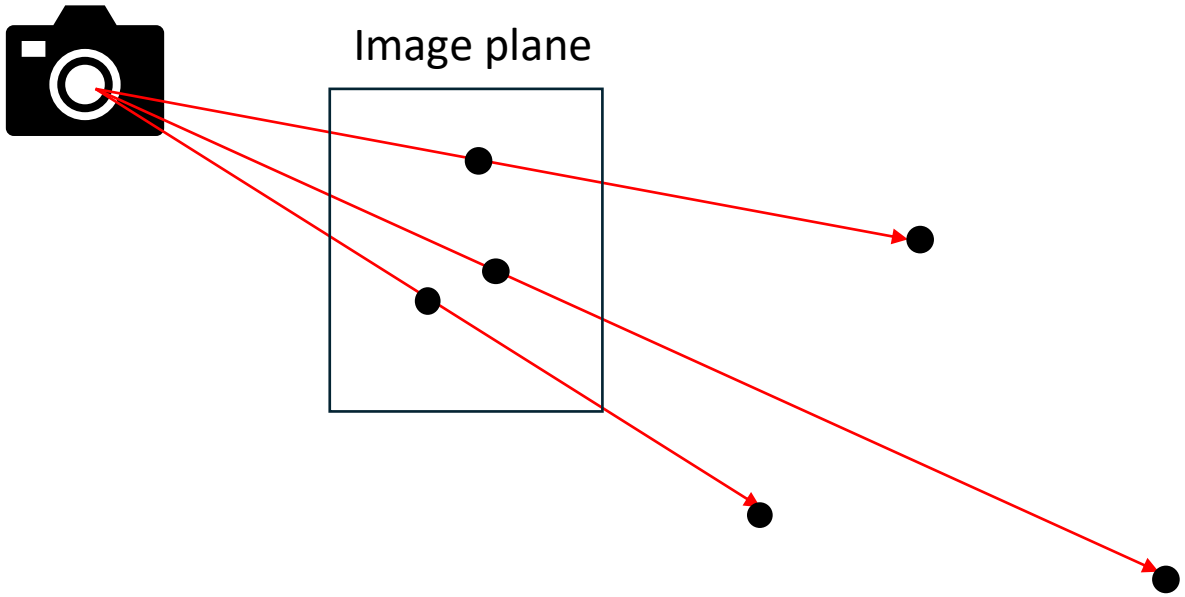
Two points is not enough to tell our location



Need a third lighthouse to localize the ship

"inscribed angle theorem"

Similar in 3D



Perspective-N-Points (PnP)

Input

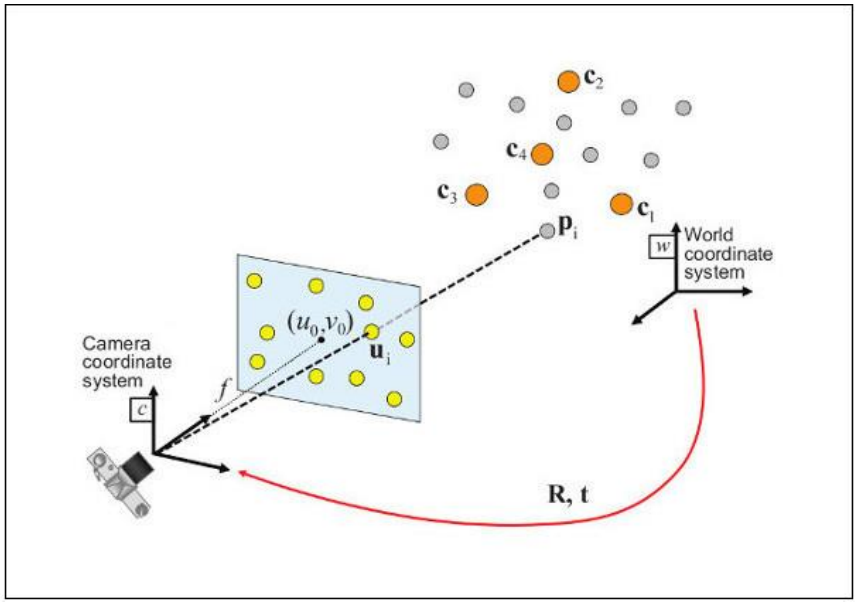
- Calibrated intrinsics $\mathbf{K}$ (or normalized image coordinates)
- n correspondences: 3D points $\boldsymbol{X}_i \in \mathbb{R}^3$ and 2D pixels $\boldsymbol{x}_i = (u_i, v_i)$

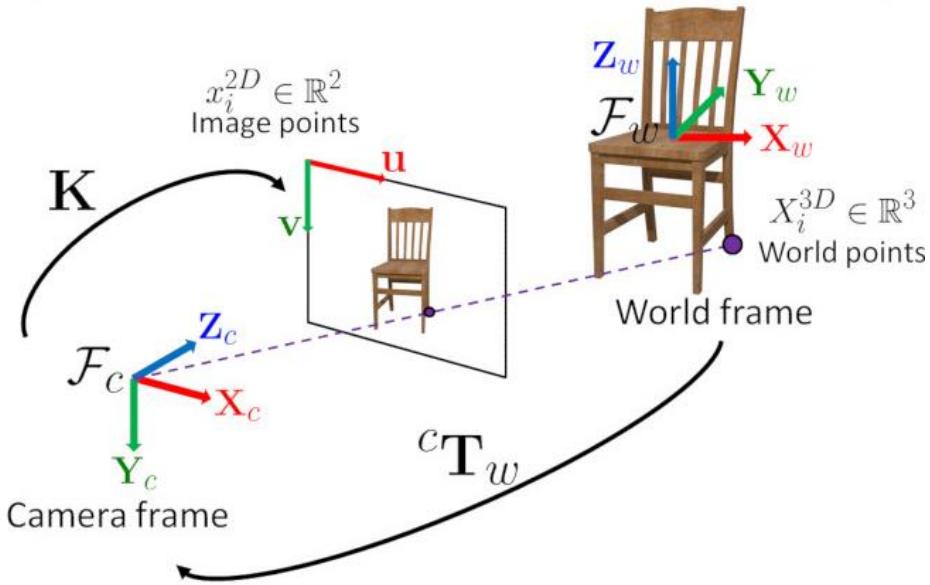
Output

A pose $(\mathbf{R}, \mathbf{t})$ that best explains the projections.

3d points in world frame. (R, t) is the pose "world to camera".

Perspective-N-Points





https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html

Solution 1: minimizing reprojection error

PnP objective: reprojection error

Define the projection function (with distortion omitted for clarity):

$$\pi\big(\mathbf{K}(\mathbf{R}\boldsymbol{X}+\mathbf{t})\big) = \begin{bmatrix} \frac{f_x X_c}{Z_c} + c_x \\ \frac{f_y Y_c}{Z_c} + c_y \end{bmatrix}, \qquad \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \mathbf{R}\boldsymbol{X} + \mathbf{t}.$$

Nonlinear least squares

$$\min_{\mathbf{R}\in SO(3),\mathbf{t}\in\mathbb{R}^3} \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \pi\big(\mathbf{K}(\mathbf{R}\boldsymbol{X}_i+\mathbf{t})\big) \right\|_2^2 .$$

Most practical solvers produce an initial estimate and then refine with Gauss–Newton / Levenberg–Marquardt.

Non-linear Least Square

We often estimate parameters $\boldsymbol{x} \in \mathbb{R}^n$ by minimizing a sum of squared residuals:

$$\min_{\boldsymbol{x}} \ \frac{1}{2}\sum_{i=1}^{m} r_i(\boldsymbol{x})^2 \qquad \Longleftrightarrow \qquad \min_{\boldsymbol{x}} \ \frac{1}{2}\|\boldsymbol{r}(\boldsymbol{x})\|^2,$$

In comparison, Linear least square always has a one-shot analytical solution

$$\min_{\boldsymbol{\theta}} \ \|\mathbf{A}\boldsymbol{\theta} - \boldsymbol{b}\|_2^2, \qquad \mathbf{A} \in \mathbb{R}^{n\times p},\ \boldsymbol{b} \in \mathbb{R}^n.$$

$$J(\boldsymbol{\theta}) = \|\mathbf{A}\boldsymbol{\theta} - \boldsymbol{b}\|_2^2 = (\mathbf{A}\boldsymbol{\theta} - \boldsymbol{b})^\top(\mathbf{A}\boldsymbol{\theta} - \boldsymbol{b}).$$

Differentiate and set to zero:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 2\mathbf{A}^\top\mathbf{A}\boldsymbol{\theta} - 2\mathbf{A}^\top\boldsymbol{b} = \mathbf{0} \quad \Rightarrow \quad \boxed{\mathbf{A}^\top\mathbf{A}\boldsymbol{\theta} = \mathbf{A}^\top\boldsymbol{b}.}$$

Normal equation

Closed-form solution (pseudo-inverse)

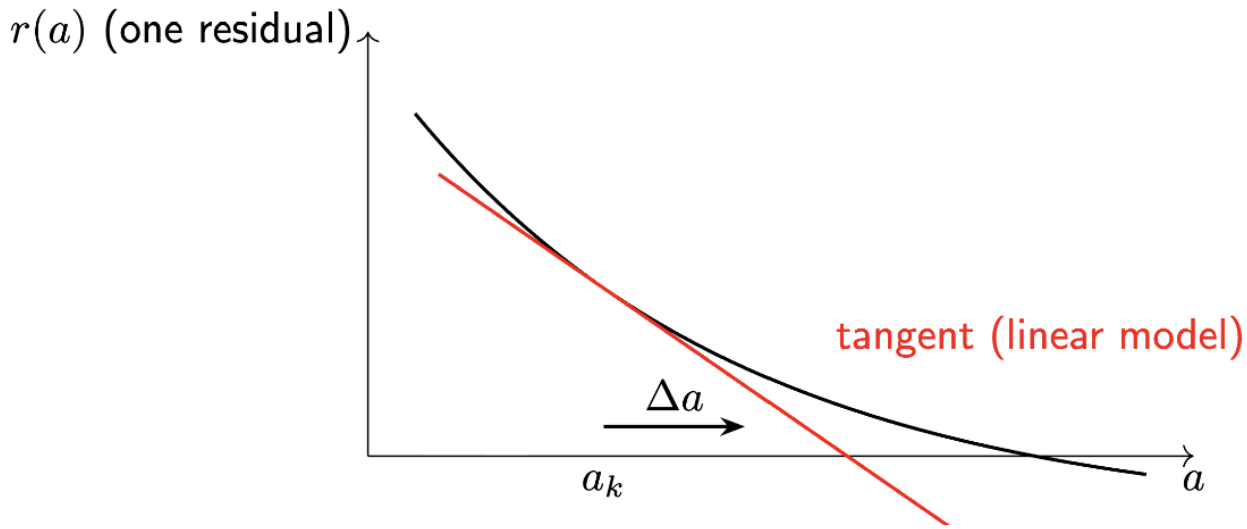
If $\mathbf{A}$ has full column rank:

$$\boldsymbol{\theta}^* = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\boldsymbol{b}.$$

Linearization

$$F(a) = \frac{1}{2}\sum_i r_i(a)^2$$

There is typically **no closed-form** minimizer for $F(a)$. r(a) is nonlinear



Linearization

At the current estimate $\boldsymbol{x}_k$, approximate residuals by first-order Taylor expansion:

$$\boldsymbol{r}(\boldsymbol{x}_k + \Delta\boldsymbol{x}) \approx \boldsymbol{r}(\boldsymbol{x}_k) + \boldsymbol{J}(\boldsymbol{x}_k)\,\Delta\boldsymbol{x}.$$

Plugging the linear model into $F(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{r}(\boldsymbol{x})\|^2$ gives a **local quadratic approximation**:

$$\boxed{F(\boldsymbol{x}_k + \Delta\boldsymbol{x}) \approx \frac{1}{2}\|\boldsymbol{r} + \boldsymbol{J}\Delta\boldsymbol{x}\|^2.}$$

Regular Linear least square

Geometric interpretation

We are choosing a step $\Delta\boldsymbol{x}$ that makes the *predicted residuals* $\boldsymbol{r} + \boldsymbol{J}\Delta\boldsymbol{x}$ as small as possible.

Solve the local least square

Solve the linearized least squares by the normal equations:

$$\boldsymbol{J}^\top \boldsymbol{J}\, \Delta \boldsymbol{x} = -\boldsymbol{J}^\top \boldsymbol{r}.$$

Gauss-Newton Method

Then update $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta \boldsymbol{x}$.

How do we get Jacobian?

- Analytical
- Numerical

Levenberg-Marquardt (LM)

LM modifies the normal equations with a damping term:

$$(\boldsymbol{J}^\top \boldsymbol{J} + \lambda \boldsymbol{I})\, \Delta \boldsymbol{x} = -\boldsymbol{J}^\top \boldsymbol{r}, \qquad \lambda \geq 0.$$

The step should not be too large

In practice

- Write out the residual function and call the non-linear optimizer

```
# Nonlinear least squares: fit y ≈ a * exp(bx)
def r(p):
    a, b = p
    x = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
    y = np.array([1.0, 1.7, 2.9, 4.8, 8.2])
    return a * np.exp(b * x) - y

# Levenberg–Marquardt (LM): method='lm' (requires #residuals >= #params)
sol_lm = least_squares(r, x0=[1.0, 1.0], method="lm")
print("LM  a,b =", sol_lm.x)
```

Converge much faster if analytical Jacobian is available

In practice

```
# Residuals: y ≈ a * exp(bx)
def r(p):
    a, b = p
    x = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
    y = np.array([1.0, 1.7, 2.9, 4.8, 8.2])
    return a * np.exp(b * x) - y

# Analytical Jacobian J_ij = ∂r_i/∂p_j
def J(p):
    a, b = p
    x = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
    e = np.exp(b * x)
    return np.column_stack([e, a * x * e])  # [∂r/∂a, ∂r/∂b]
```

One Caveat

Nonlinear least squares

$$\min_{\mathbf{R}\in SO(3),\mathbf{t}\in\mathbb{R}^3} \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \pi\big(\mathbf{K}(\mathbf{R}\boldsymbol{X}_i + \mathbf{t})\big) \right\|_2^2 .$$

Most practical solvers produce an initial estimate and then refine with Gauss–Newton / Levenberg–Marquardt.

Need special care to make R valid

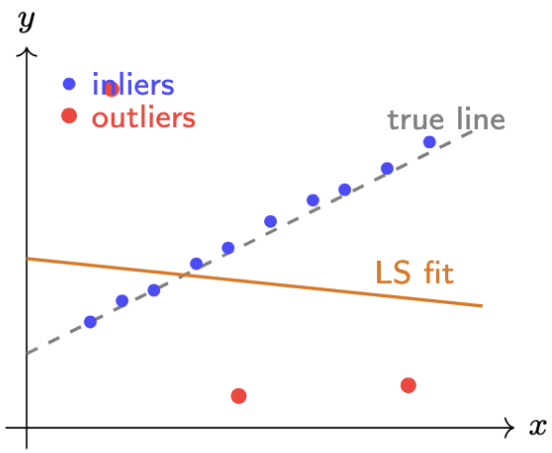
Issue: outliers and initialization

- Outliers introduce errors
- Bad initialization prevents convergence

Classic least squares

Fit a line $y = ax + b$ by minimizing

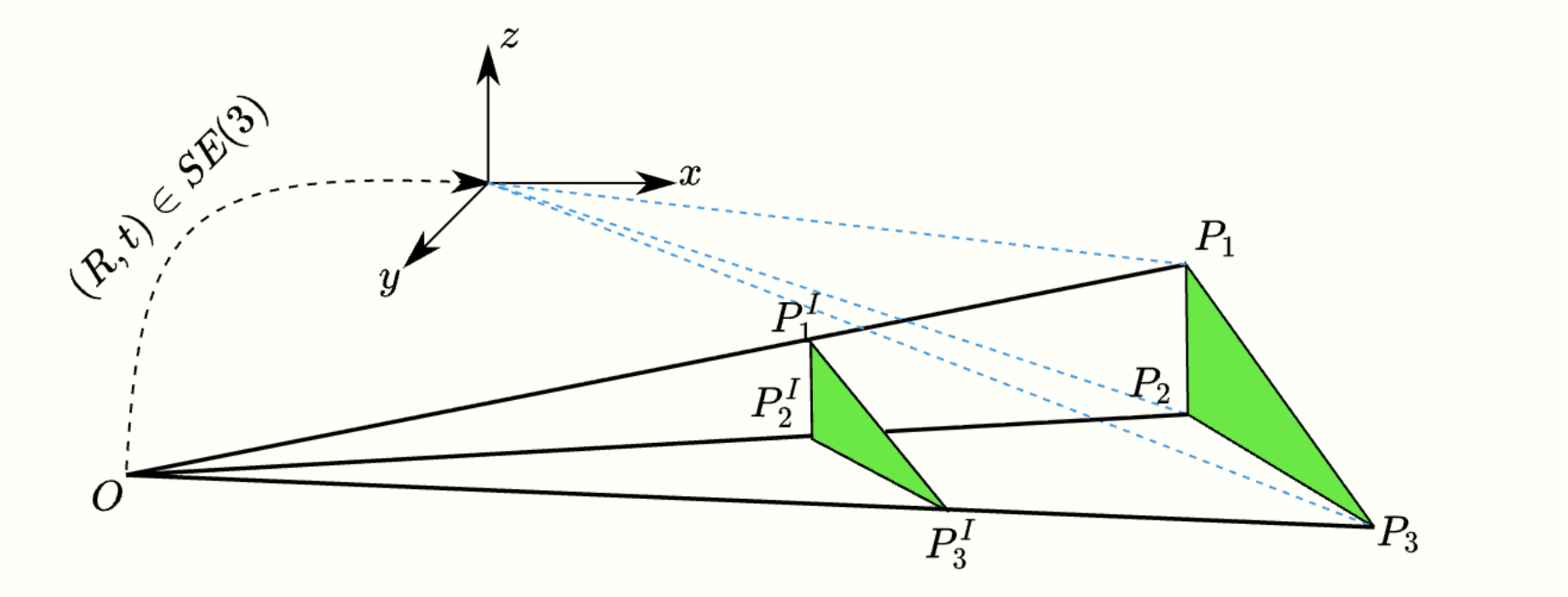
$$\min_{a,b} \sum_i \left(y_i - (ax_i + b)\right)^2.$$



Solution 2: Minimal solver + outlier rejection

- We need 3 points to solve PnP
  - 4 solutions
  - PnP -> P3P
- <u>One extra point</u> to get a unique solution

P3P (Perspective-3-point) problem



https://jingnanshi.com/blog/pnp_minimal.html

P3P: step 1 – normalize to rays

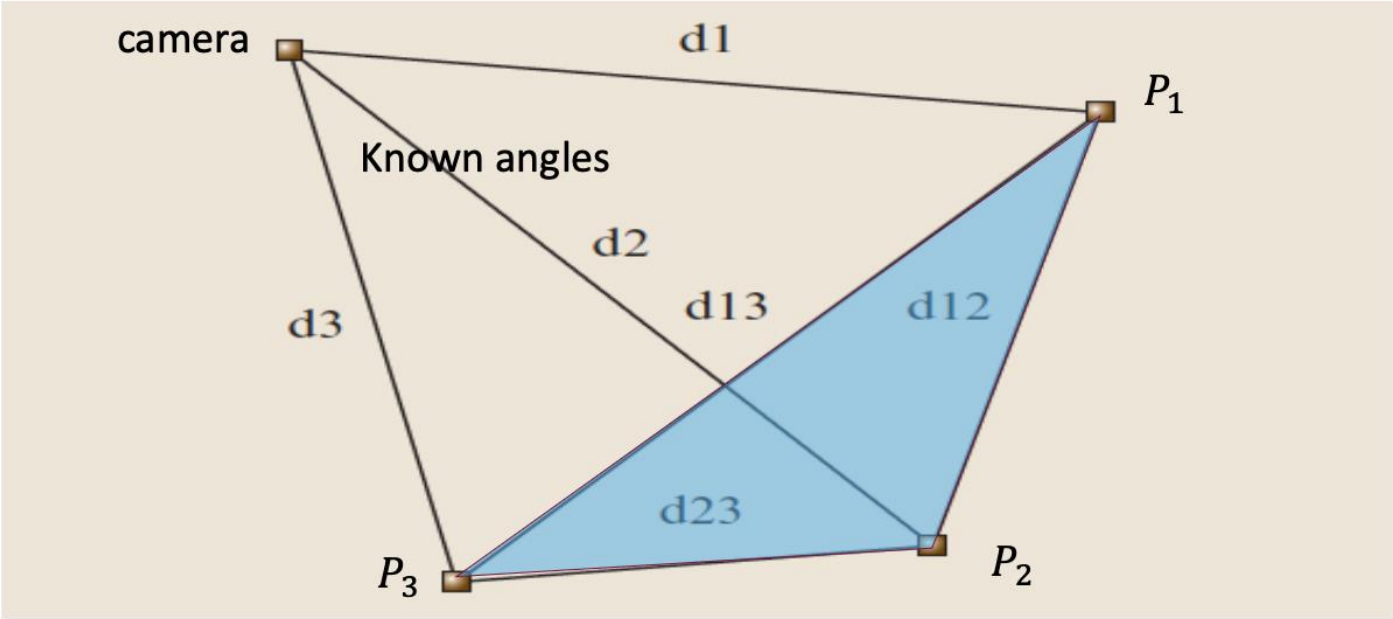
We are given 3 correspondences $(\boldsymbol{X}_i, \boldsymbol{x}_i)$ for $i \in \{1, 2, 3\}$.

Normalize image rays

Convert pixels to normalized coordinates:

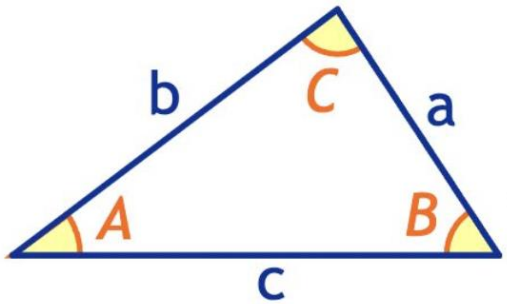
$$\hat{\boldsymbol{x}}_i \sim \mathbf{K}^{-1} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}, \qquad \boldsymbol{d}_i = \frac{\hat{\boldsymbol{x}}_i}{\|\hat{\boldsymbol{x}}_i\|} \in \mathbb{R}^3.$$

P3P: step 2 – trigonometry



Cosine Law

For any triangle:



a, **b** and **c** are sides.

C is the angle opposite side c

The Law of Cosines (also called the **Cosine Rule**) says:

$$c^2 = a^2 + b^2 - 2ab\cos(C)$$

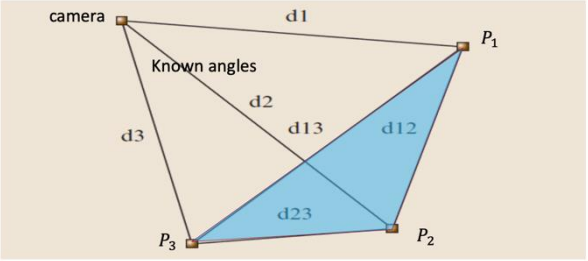
Triangle Cosine Law

Let world distances between the three points be

$$d_{23} = \|\boldsymbol{P}_2 - \boldsymbol{P}_3\|, \quad d_{13} = \|\boldsymbol{P}_1 - \boldsymbol{P}_3\|, \quad d_{12} = \|\boldsymbol{P}_1 - \boldsymbol{P}_2\|.$$

Let angles between rays be

$$\alpha = \angle(\boldsymbol{u}_2, \boldsymbol{u}_3), \quad \beta = \angle(\boldsymbol{u}_1, \boldsymbol{u}_3), \quad \gamma = \angle(\boldsymbol{u}_1, \boldsymbol{u}_2).$$



Law of cosines (in the camera-centered triangle)

With unknown distances d_1, d_2, d_3:

$$d_2^2 + d_3^2 - 2d_2d_3\cos\alpha = d_{23}^2,$$
$$d_1^2 + d_3^2 - 2d_1d_3\cos\beta = d_{13}^2,$$
$$d_1^2 + d_2^2 - 2d_1d_2\cos\gamma = d_{12}^2.$$

Solve for d1, d2, and d3

P3P: Step 4 - solve by elimination

Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem

ROBERT M. HARALICK
Intelligent Systems Laboratory, Department of Electrical Engineering FT-10, University of Washington, Seattle, WA 98195, USA

CHUNG-NAN LEE
Institute of Information Engineering, National Sun Yat-Sen University, Kaohsiung, Taiwan 80424, ROC

KARSTEN OTTENBERG
Philips-Forschungslabor, Vogt-Kölln Straße 30, D-2000 Hamburg 54, Germany

MICHAEL NÖLLE
Technische Universität Hamburg-Harburg, Technische Informatik I, Harburger Schloßstraße 20, D-2100 Hamburg 90, Germany

Received October 30, 1990; revised March 27, 1992 and October 29, 1993

Equation (1) to (9), solving a fourth order polynomial

P3P Solutions and Ambiguity

The P3P polynomial system generally yields multiple valid configurations.

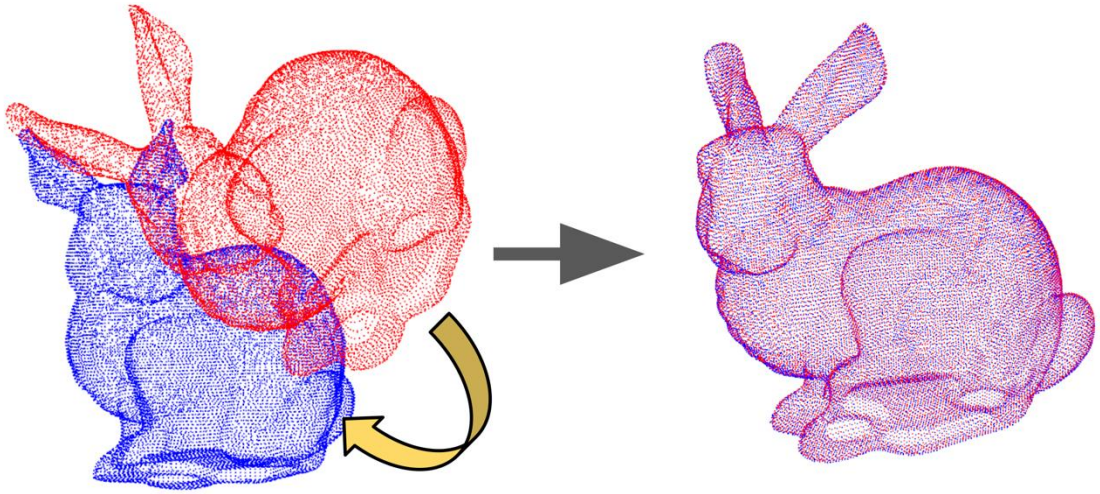
- Up to **4** real solutions for (d_1, d_2, d_3).
- Each solution implies 3 camera-frame points: $\boldsymbol{p}_i = d_i \, \boldsymbol{u}_i$.
- Then recover $(\mathbf{R}, \mathbf{t})$ aligning world points $\boldsymbol{P}_i$ to camera points $\boldsymbol{p}_i$.

Reduces to 3D-to-3D registration problem

3D to 3D registration (Procrustes problem)

Find the rigid transform $(\mathbf{R}, \mathbf{t})$ that best aligns the two sets:

$$\min_{\mathbf{R} \in SO(3),\, \mathbf{t} \in \mathbb{R}^3} \sum_i \|\boldsymbol{p}_i - (\mathbf{R}\boldsymbol{P}_i + \mathbf{t})\|_2^2.$$



Will come back to this next time

Disambiguating: use a 4th point

To choose the correct solution among multiple P3P candidates:

Most common strategy

- Evaluate each candidate pose on an additional correspondence $(\boldsymbol{P}_4, \boldsymbol{x}_4)$.
- Compute reprojection error; pick the solution with smallest error and positive depth.

Full pipeline: P3P Solver

1. **Pixels → rays:** $\boldsymbol{u}_i \propto \mathbf{K}^{-1}[u_i \; v_i \; 1]^\top$, normalize $\|\boldsymbol{u}_i\| = 1$.
2. **Solve depths:** use cosine-law constraints with $d_{ij} = \|\boldsymbol{P}_i - \boldsymbol{P}_j\|$ and $c_{ij} = \boldsymbol{u}_i^\top \boldsymbol{u}_j$ to get (d_1, d_2, d_3).
3. **Lift to 3D:** $\boldsymbol{p}_i = d_i \boldsymbol{u}_i$.
4. **Pose via Procrustes:** find $(\mathbf{R}, \mathbf{t})$ s.t. $\boldsymbol{p}_i \approx \mathbf{R}\boldsymbol{P}_i + \mathbf{t}$.
5. **Disambiguate:** choose candidate $(\mathbf{R}^{(k)}, \mathbf{t}^{(k)})$ with cheirality + smallest reprojection error (use $(\boldsymbol{P}_4, \boldsymbol{x}_4)$ if available).

Summary

- PnP can be solved with non-linear least square
- PnP can be solved with P3P and RANSAC
  - P3P is the minimal problem (4 points for unique solution)